Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «ИЗБЫТОЧНОЕ КОДИРОВАНИЕ ДАННЫХ • В ИНФОРМАЦИОННЫХ СИСТЕМАХ. КОД ХЕММИНГА. ИТЕРАТИВНЫЕ КОДЫ»

Выполнил:
Студент 4-го курса
Группы АС-65
Гуща И.В.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга, итеративного кода

Задание1. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 или 4.

Вариант 3.

```
def coder(file_name):
    data = read file(file name)
    encoded = bytearray()
    buffer = []
    for byte in data:
        bits = [(byte >> i) & 1 for i in range(7, -1, -1)]
        groups = [
            bits[:4],
            bits[4:]
        ]
        for group in groups:
            encoded_bits = np.array(group) @ G
            encoded_bits = encoded_bits % 2
            buffer.extend(encoded_bits.tolist())
            while len(buffer) >= 8:
                byte_bits = buffer[:8]
                buffer = buffer[8:]
                value = 0
                for bit in byte_bits:
                    value = (value << 1) | bit
                encoded.append(value)
    if len(buffer) > 0:
        buffer.extend([0] * (8 - len(buffer)))
        value = 0
        for bit in buffer:
            value = (value << 1) | bit
        encoded.append(value)
    write_file("encoded.txt", encoded)

def syndrome(code):
    code = np.array(code)
    s = code @ H
    s = s % 2

    print("7 бит:       ", code)
    print("Синдром:     ", s)

    # Если синдром не равен 000
    # значит есть ошибка
    if np.any(s):
        for i in range(7):
            if np.array_equal(H[i], s):
                print("Ошибка в бите:", i + 1)

                # Исправляем ошибочный бит
                code[i] ^= 1

                break

    information_bits = code[:4]

    print("Исправленные: ", code)
    print("4 инф. бита:  ", information_bits)
```

```
    print()

    return information_bits.tolist()

def decoder():
    data = read_file("encoded.txt")

    decoded = bytearray()
    buffer = []

    for byte in data:

        # Превращаем байт в 8 бит
        bits = [(byte >> i) & 1 for i in range(7, -1, -1)]

        # Добавляем биты в буфер
        buffer.extend(bits)

        # Из буфера берём по 14 бит
        while len(buffer) >= 14:

            block = buffer[:14]
            buffer = buffer[14:]
            group1 = block[:7]
            group2 = block[7:]
            info1 = syndrome(group1)
            info2 = syndrome(group2)
            decoded_bits = info1 + info2

            print("Декодированные 8 бит:", decoded_bits)
            print()
            value = 0

            for bit in decoded_bits:
                value = (value << 1) | bit

            decoded.append(value)

    write_file("decoded.txt", decoded)
```

Исходный текст находиться в файле text.txt и содержит данные «pyz».
При запуске его с 0 ошибок он успешно декодируется, синдром во всех случаях равен 0.
При запуске с 1 гарантированной ошибкой в каждом байте происходит следуещее:

```
coder("text.txt")
pomeha("encoded.txt",1,1)
decoder()
```

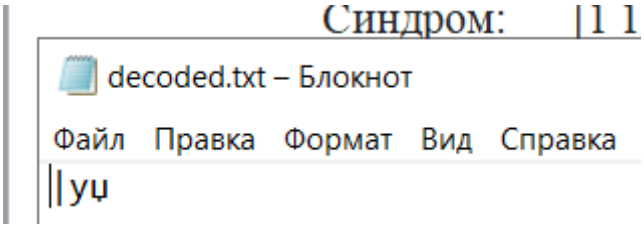
Вывод программы:
7 бит: [0 1 1 0 0 0 1]
Синдром: [1 1 1]
Ошибка в бите: 4
Исправленные: [0 1 1 1 0 0 1]
4 инф. бита: [0 1 1 1]
7 бит: [0 0 1 0 0 0 0]
Синдром: [0 1 1]
Ошибка в бите: 3
Исправленные: [0 0 0 0 0 0 0]
4 инф. бита: [0 0 0 0]
Декодированные 8 бит: [0, 1, 1, 1, 0, 0, 0, 0]

7 бит: [0 1 1 0 0 0 1]
Синдром: [1 1 1]

Ошибка в бите: 4
Исправленные: [0 1 1 1 0 0 1]
4 инф. бита: [0 1 1 1]

7 бит: [1 0 0 1 1 0 1]
Синдром: [1 0 0]
Ошибка в бите: 5
Исправленные: [1 0 0 1 0 0 1]
4 инф. бита: [1 0 0 1]
Декодированные 8 бит: [0, 1, 1, 1, 1, 0, 0, 1]

В результате исходный файл совпадает с декодированным.
При наличии 2 гарантированных ошибок, файл не совпадает с исходным. Несмотря на то, что программа исправила ошибки по указанным синдромам, в реальности только внесло новые изменения. :



При использовании кода Хэминга 4, при наличии 2 ошибок в кодовом слове.

```
coder4("text.txt")
pomeha("encoded4.txt",2,1)
decoder4()
```

```
    raise ValueError(
ValueError: Обнаружена двойная ошибка. Кодовое слово нельзя надёжно декодировать.
```

С одной ошибкой информация декодируется правильно.
Задание2: Разработать собственное приложение, которое позволяет вы- полнять следующие операции: 1) вписывать произвольное двоичное представление инфор мационного слова Хк (кодируемой информации) длиной К битов в двумерную матрицу размерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2); 2) вычислять проверочные биты (биты паритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов); 3) формировать кодовое слово Х, присоединением избыточ- ных символов к информационному слову; 4) генерировать ошибку произвольной кратности (i, i > О), распределенную случайным (фазом среди символов слова Х, в результате чего формируется кодовое слово Уп; 5) определять местоположение ошибочных символов итера- тивным кодом в слове Уп в соответствии с исполызусмыми груп- пами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления слово Уп '); б) выполнять анализ корректирующей способности исполь- зуемого кода (количественная оценка) путем сравнения соответст- вующих слов Х, и У„' ; результат анализа может быть представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ошибки этой кратности, которые правильно обнаружены и кото- рые правильно скорректированы.

```
def encode(info, mode):
    data_groups, names = make_groups(mode)

    code = info.copy()
    groups = []
    parity_positions = []

    for group in data_groups:
        total = 0
```

```
            for position in group:
                total += code[position]

            parity = total % 2

            parity_position = len(code)
            code.append(parity)
            full_group = group.copy()
            full_group.append(parity_position)
            groups.append(full_group)
            parity_positions.append(parity_position)
        total = 0

        for bit in code:
            total += bit

        overall_parity = total % 2
        overall_position = len(code)
        code.append(overall_parity)
        overall_group = []

        for i in range(len(code)):
            overall_group.append(i)

        groups.append(overall_group)
        names.append("Общий паритет")

        return code, groups, names, parity_positions, overall_position

    def print_code_table(code, mode, parity_positions,
                         overall_position):
        matrix = make_matrix(code[:K])

        print("\nТАБЛИЦА КОДОВОГО СЛОВА")
        print("-" * 53)
        print("       Информационные биты       | Rстр")

        for r in range(ROWS):
            print(
                " ".join(str(bit) for bit in matrix[r]),
                "|", code[parity_positions[r]]
            )

        print("-" * 53)

        print("Rстолб:", end=" ")

        for c in range(COLS):
            position = parity_positions[ROWS + c]
            print(code[position], end=" ")

        print()

        if mode == 3:
            print("\nПроверочные биты диагоналей:")

            for i in range(ROWS + COLS, len(parity_positions)):
                position = parity_positions[i]

                print(
                    "P" + str(i - ROWS - COLS + 1)
                    + " = " + str(code[position]),
                    end="   "
                )
```

```
            print()

        print(
            "\nОбщий бит паритета Робщ =",
            code[overall_position]
        )

        print("\nПолное кодовое слово:")
        print(" ".join(str(bit) for bit in code))
        print("Длина кодового слова:", len(code))

    def get_syndrome(code, groups):
        syndrome = []
        for i in range(len(groups)):
            total = 0

            for position in groups[i]:
                total += code[position]

            if total % 2 != 0:
                syndrome.append(i)

        return syndrome


    def make_errors(code, count=None, positions=None):
        result = code.copy()

        if positions is not None:
            # Пользователь вводит позиции начиная с 1
            selected = []
            for position in positions:
                if position < 1 or position > len(result):
                    raise ValueError(
                        "Позиция вне диапазона кодового слова."
                    )
                index = position - 1
                if index in selected:
                    raise ValueError(
                        "Одна позиция указана несколько раз."
                    )
                selected.append(index)
        else:
            if count is None:
                raise ValueError(
                    "Укажите количество ошибок или позиции."
                )

            if count < 1 or count > len(result):
                raise ValueError(
                    "Недопустимое количество ошибок."
                )

            selected = random.sample(range(len(result)), count)

        for position in selected:
            result[position] = 1 - result[position]
        positions_used = []

        for position in selected:
            positions_used.append(position + 1)

        return result, positions_used
```

```
def correct(code, groups):
    result = code.copy()

    for step in range(len(result)):
        syndrome = get_syndrome(result, groups)

        if len(syndrome) == 0:
            break

        found = False

        # Ищем бит, входящий во все нарушенные группы
        for position in range(len(result)):
            signature = []

            for i in range(len(groups)):
                if position in groups[i]:
                    signature.append(i)

            if signature == syndrome:
                result[position] = 1 - result[position]
                found = True
                break

        if not found:
            break

    return result
```

```
Исходное кодовое слово X:

ТАБЛИЦА КОДОВОГО СЛОВА
-----------------------------------------------------
       Информационные биты       | Рстр
1 0 1 1 0 0 | 1
0 1 1 0 1 0 | 1
1 1 0 0 1 0 | 1
0 1 1 0 0 1 | 1
-----------------------------------------------------
Рстолб: 0 1 1 1 0 1

Проверочные биты диагоналей:
P1 = 0   P2 = 0   P3 = 0   P4 = 0   P5 = 1   P6 = 1   P7 = 0   P8 = 0   P9 =
0

Общий бит паритета Робщ = 0

Полное кодовое слово:
1 0 1 1 0 0 0 1 1 0 1 0 1 1 0 0 1 0 0 1 1 0 0 1 1 1 1 1 0 1 1 1 0 1 0 0 0 0 1
1 0 0 0 0
Длина кодового слова: 44
Полученное слово Y с ошибкой:
10110000101011001001100111110111010000110000

Исправленное кодовое слово Y':
10110001101011001001100111110111010000110000

Таблица исправленного слова:

ТАБЛИЦА КОДОВОГО СЛОВА
-----------------------------------------------------
       Информационные биты       | Рстр
1 0 1 1 0 0 | 1
```

```
0 1 1 0 1 0 | 1
1 1 0 0 1 0 | 1
0 1 1 0 0 1 | 1
-----------------------------------------------------
Рстолб: 0 1 1 1 0 1

Проверочные биты диагоналей:
P1 = 0   P2 = 0   P3 = 0   P4 = 0   P5 = 1   P6 = 1   P7 = 0   P8 = 0   P9 =
0

Общий бит паритета Робщ = 0

Полное кодовое слово:
1 0 1 1 0 0 0 1 1 0 1 0 1 1 0 0 1 0 0 1 1 0 0 1 1 1 1 1 0 1 1 1 0 1 0 0 0 0 1
1 0 0 0 0
Длина кодового слова: 44

Результат проверки:
Кодовое слово восстановлено правильно.
Количество отличающихся битов: 0

Восстановленная информационная часть:
101100011010110010011001
Информационное слово восстановлено правильно.
```

Вывод : при передаче информации существует риск ее искажения или изменения. При этом использование дополнительных бит позволяет повысить безопасность передачи. Такое шифрование называется избыточным. Примерами такого шифрования является код Хэмминга и итеративный.